

PostgreSQL Транзакции - Практические упражнения

Подготовка базы данных

```
CREATE TABLE accounts (  
    account_id SERIAL PRIMARY KEY,  
    balance NUMERIC NOT NULL  
);  
  
INSERT INTO accounts (balance) VALUES (1000), (1500), (2000);
```

Упражнение 1

Создать атомарную транзакцию для перевода \$200 с аккаунта 1 на аккаунт 2

```
BEGIN;  
  
SELECT balance FROM accounts WHERE account_id = 1 FOR UPDATE;  
  
UPDATE accounts  
SET balance = balance - 200  
WHERE account_id = 1  
    AND balance >= 200;  
  
UPDATE accounts  
SET balance = balance + 200  
WHERE account_id = 2;  
  
COMMIT;
```

Упражнение 2

Обработать попытку перевода \$5000 при недостаточном балансе

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 5000  
WHERE account_id = 1;  
  
UPDATE accounts  
SET balance = balance + 5000
```

```
WHERE account_id = 3;

-- Проверяем не ушел ли баланс в минус
IF (SELECT balance FROM accounts WHERE account_id = 1) < 0 THEN
    ROLLBACK;
ELSE
    COMMIT;
END IF;
```

Упражнение 3

Показать работу уровня изоляции SERIALIZABLE

```
-- Транзакция 1:
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT balance FROM accounts WHERE account_id = 1;
UPDATE accounts SET balance = 500 WHERE account_id = 1;
COMMIT;

-- Транзакция 2 (запустить параллельно):
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT balance FROM accounts WHERE account_id = 1;
UPDATE accounts SET balance = 800 WHERE account_id = 1;
COMMIT;
```

Обработка PostgreSQL:

- При попытке COMMIT второй транзакции получим ERROR: could not serialize access due to concurrent update
- Одна из транзакций будет прервана
- Необходимо повторить прерванную транзакцию

Упражнение 4

Использовать savepoints для частичного отката

```
BEGIN;

INSERT INTO accounts (balance) VALUES (500);

SAVEPOINT my_savepoint;

UPDATE accounts SET balance = balance + 200 WHERE account_id = 1;
UPDATE accounts SET balance = balance - 200 WHERE account_id = 999; --
Ошибка
```

```
ROLLBACK TO my_savepoint;  
  
COMMIT;
```

Упражнение 5

Создать и разрешить ситуацию взаимной блокировки

Сценарий взаимной блокировки в PostgreSQL

Сессия 1:

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

Сессия 2:

```
BEGIN;  
UPDATE accounts SET balance = balance - 50 WHERE account_id = 2;
```

Сессия 1:

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;  
-- Блокировка: ждет завершения UPDATE из сессии 2
```

Сессия 2:

```
UPDATE accounts SET balance = balance + 50 WHERE account_id = 1;  
-- PostgreSQL обнаружит взаимную блокировку и завершит одну транзакцию  
-- Ошибка: ERROR: deadlock detected
```

PostgreSQL автоматически:

1. Обнаружит цикл блокировок
2. Прервет одну транзакцию (обычно ту, что дешевле откатить)
3. Выдаст ошибку **deadlock detected** в прерванной сессии
4. Другая транзакция продолжит работу

Упражнение 6

```
CREATE TABLE transfers (  
  id SERIAL PRIMARY KEY,  
  from_account_id INTEGER REFERENCES accounts(id),  
  to_account_id INTEGER REFERENCES accounts(id),  
  amount DECIMAL(10, 2)  
);
```

Перевод денег с записью в историю транзакций

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 200  
WHERE account_id = 1;  
  
UPDATE accounts  
SET balance = balance + 200  
WHERE account_id = 2;  
  
INSERT INTO transfers (from_account_id, to_account_id, amount)  
VALUES (1, 2, 200);  
  
COMMIT;
```